Classification of Iris Flowers and Handwritten Digits

TTT4275 Estimation, Detection and Classification — NTNU

Student A Student B

April 22, 2026

Abstract

This report presents the two classification projects assigned in TTT4275. The first project is the mandatory Iris task, in which a linear classifier is trained by minimizing the mean squared error (MSE) via iterative gradient descent. The effect of the training/test split on generalization is investigated, and the relative importance of the four features is analysed by progressively removing the least discriminative feature. The second project addresses classification of handwritten digits from the MNIST database using variants of the nearest-neighbour classifier described in chapter 3 of the course compendium. A 1-NN classifier built from all 60 000 training images is compared with 1-NN and 7-NN classifiers built from a compact template set produced by per-class k-means clustering. Both projects follow the constructions given in the compendium, and the code is provided in the appendices.

Contents

1 Introduction **2**

2 Theory **2**
2.1 Linear Discriminant . . . 2
2.2 Sigmoid Activation . . . 2
2.3 MSE Training Criterion . . . 2
2.4 Gradient and Update . . . 3
2.5 Nearest-Neighbour and K-Nearest-Neighbour Classifiers . . . 3
2.6 Template Clustering . . . 3
2.7 Confusion Matrix and Error Rate . . . 4

3 The Task **4**

4 Iris — Implementation and Results **4**
4.1 Implementation Overview . . . 5
4.2 Task 1: Training and Generalization . . . 5
4.3 Task 2: Features and Linear Separability . . . 6

5 MNIST — Implementation and Results **8**
5.1 Implementation Overview . . . 8
5.2 Task 1: NN Classifier with 60 000 Templates . . . 8
5.3 Task 2: Clustered Templates, NN and KNN . . . 10
5.4 Comparison of the Three Systems . . . 11

6 Conclusion **11**

A MATLAB Code — Iris **12**
A.1 Task 1: Training and Generalization 12
A.2 Task 2: Features and Linear Separability 14

B MATLAB Code — MNIST **17**
B.1 Task 1: NN classifier with the full training set 17
B.2 Task 2: Clustered templates, NN and KNN 19

1 Introduction

Pattern classification is a central problem in signal processing and machine learning, where the goal is to assign an observation to one of several predefined categories based on measured features. It has broad applications ranging from medical diagnosis to speech recognition and autonomous systems.

This report covers the two classification projects assigned in TTT4275. The first is the Iris task, which uses the classical Fisher Iris dataset [1] of 150 flower samples from three species described by four measurements each. The dataset is close to linearly separable and is used to train and evaluate the linear MSE classifier of chapters 2.4–3.2 of the compendium [2]. The second project is the MNIST handwritten-digit classification task. Here a 28×28-pixel image is treated as a 784-dimensional feature vector and classified with the nearest-neighbour (NN) and K-nearest-neighbour (KNN) classifiers of chapter 3, applied both to the full 60 000-image training set and to a compact template set obtained by per-class k-means clustering.

The rest of this report is organized as follows. Section 2 presents the theoretical background on the linear MSE classifier and the nearest-neighbour classifiers used later. Section 3 describes the specific tasks. Section 4 covers the Iris implementation and results, and Section 5 covers the MNIST implementation and results. Section 6 summarizes the main findings.

2 Theory

This section presents the theory behind the linear MSE classifier used in this project. The derivations follow the course compendium [2], chapters 2.4 and 3.2.

2.1 Linear Discriminant

A linear discriminant classifier describes each class ω_i by a linear function (compendium Eq. 7)

$$g_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} + w_{i0}, \qquad i = 1, \ldots, C. \tag{1}$$

Stacking the bias into the weight matrix $\mathbf{W} \in \mathbb{R}^{C \times (D+1)}$ and augmenting the feature vector as $\tilde{\mathbf{x}} = [\mathbf{x}^T \; 1]^T$ gives the compact form $\mathbf{z} = \mathbf{W}\tilde{\mathbf{x}}$. A sample $\mathbf{x}$ is assigned to the class $\hat{c}$ corresponding to the largest discriminant value (Eq. 6):

$$\hat{c} = \arg\max_i \; g_i(\mathbf{x}). \tag{2}$$

2.2 Sigmoid Activation

Since the one-hot targets take binary values $\{0, 1\}$, the raw discriminant $\mathbf{z} = \mathbf{W}\tilde{\mathbf{x}}$ is passed through a smooth sigmoid (compendium Eq. 20):

$$g_{ik} \;=\; \mathrm{sigmoid}(z_{ik}) \;=\; \frac{1}{1 + e^{-z_{ik}}}, \qquad i = 1, \ldots, C. \tag{3}$$

The sigmoid is used because the gradient-descent training in the next subsection requires a differentiable approximation of the Heaviside step; since it is monotonic, it does not change the argmax decision in Eq. (2).

2.3 MSE Training Criterion

The weight matrix $\mathbf{W}$ is found by minimizing the mean squared error between the classifier outputs and the one-hot targets $\mathbf{t}_k \in \{0, 1\}^C$ (compendium Eq. 19):

$$\mathrm{MSE} \;=\; \tfrac{1}{2} \sum_{k=1}^{N} \big(\mathbf{g}_k - \mathbf{t}_k\big)^T \big(\mathbf{g}_k - \mathbf{t}_k\big). \tag{4}$$

2.4 Gradient and Update

Because the sigmoid is smooth, the gradient of the MSE in Eq. (4) can be obtained through the chain rule. Using $\nabla_{\mathbf{g}_k}\mathrm{MSE} = \mathbf{g}_k - \mathbf{t}_k$, the sigmoid derivative $\nabla_{\mathbf{z}_k}\mathbf{g}_k = \mathbf{g}_k \circ (\mathbf{1} - \mathbf{g}_k)$ and $\nabla_{\mathbf{W}}\mathbf{z}_k = \mathbf{x}_k^T$, one obtains (compendium Eq. 22):

$$\nabla_{\mathbf{W}}\mathrm{MSE} \;=\; \sum_{k=1}^{N} \big[(\mathbf{g}_k - \mathbf{t}_k) \circ \mathbf{g}_k \circ (\mathbf{1} - \mathbf{g}_k)\big]\, \mathbf{x}_k^T, \tag{5}$$

where $\circ$ denotes elementwise multiplication. The weights are updated in the opposite direction of the gradient (compendium Eq. 23):

$$\mathbf{W}(m) \;=\; \mathbf{W}(m-1) - \alpha\, \nabla_{\mathbf{W}}\mathrm{MSE}, \tag{6}$$

where $\alpha > 0$ is the step factor, which is chosen by trial and error so that the MSE decreases monotonically without oscillating.

2.5 Nearest-Neighbour and K-Nearest-Neighbour Classifiers

The compendium describes in chapter 3 a complementary family of classifiers that do not estimate a parametric decision surface but store a set of labelled reference vectors ("templates") $\{\mathbf{t}_j, c_j\}_{j=1}^{N_T}$. For a new sample $\mathbf{x}$ the Euclidean distance

$$d(\mathbf{x}, \mathbf{t}_j) \;=\; \|\mathbf{x} - \mathbf{t}_j\|_2 \;=\; \sqrt{(\mathbf{x} - \mathbf{t}_j)^T(\mathbf{x} - \mathbf{t}_j)} \tag{7}$$

is computed to every template, and the *nearest-neighbour* (NN) rule assigns the class of the closest template:

$$\hat{c}_{\mathrm{NN}}(\mathbf{x}) \;=\; c_{j^\star}, \qquad j^\star = \arg\min_j \; d(\mathbf{x}, \mathbf{t}_j). \tag{8}$$

The K*-nearest-neighbour* (KNN) rule is a robust generalization: the K closest templates $\{\mathbf{t}_{j_1}, \ldots, \mathbf{t}_{j_K}\}$ are located and $\mathbf{x}$ is assigned to the class that occurs most frequently among their labels:

$$\hat{c}_{\mathrm{KNN}}(\mathbf{x}) \;=\; \mathrm{mode}\big(c_{j_1}, \ldots, c_{j_K}\big). \tag{9}$$

When N_T is very large (60 000 MNIST images) the classifier is slow because the full $N_{\mathrm{test}} \times N_T$ distance matrix must be evaluated. For computational efficiency the distances are computed in chunks using the identity

$$\|\mathbf{x} - \mathbf{t}\|_2^2 \;=\; \|\mathbf{x}\|_2^2 + \|\mathbf{t}\|_2^2 - 2\,\mathbf{x}^T\mathbf{t}, \tag{10}$$

so that the cross-term reduces to a single matrix product.

2.6 Template Clustering

The KNN classifier can be made faster and more robust by replacing the 60 000 training images with a small set of *prototypes* found by clustering. For each class ω_i separately, the training vectors are grouped into M clusters by k-means, which iteratively minimizes the within-cluster sum of squares

$$J \;=\; \sum_{m=1}^{M} \sum_{\mathbf{x}_k \in \mathcal{S}_m} \|\mathbf{x}_k - \mathbf{c}_m\|_2^2, \tag{11}$$

where $\mathcal{S}_m$ is the set of training vectors assigned to cluster m and $\mathbf{c}_m$ is the corresponding centroid. The $C \cdot M$ centroids are then used as templates in Eqs. (8)/(9).

2.7 Confusion Matrix and Error Rate

Classifier performance is evaluated using a confusion matrix, which is a $C \times C$ matrix where element (i, j) counts the number of samples from true class i that were classified as class j. The overall error rate is defined as

$$\text{Error rate} = \frac{\text{Number of misclassified samples}}{\text{Total number of samples}} \times 100\%. \tag{12}$$

3 The Task

This section describes the tasks addressed in this report. The project assignment offers a choice between multiple classification problems; the Iris task is mandatory for all groups, and we have additionally chosen the MNIST handwritten-digit task.

Iris

The Iris dataset contains 50 samples from each of three flower species (Setosa, Versicolor, Virginica), with four features per sample (sepal length, sepal width, petal length, petal width). Two sub-tasks are performed:

Task 1 — Training and generalization. A linear MSE classifier is trained and tested using two different train/test splits: first using the first 30 samples per class for training and the last 20 for testing, then swapping so that the last 30 are used for training and the first 20 for testing. Confusion matrices and error rates are computed for both the training and test sets in each case, and the results are compared.

Task 2 — Features and linear separability. Using the first train/test split from Task 1, histograms of each feature are plotted per class. The feature showing the most overlap between classes is identified and removed. The classifier is then retrained and tested with the remaining three features. This process is repeated, removing one more feature at each step, resulting in classifiers using 4, 3, 2, and 1 features. The confusion matrices and error rates are compared to assess the relative importance of each feature for linear separability.

MNIST

The MNIST database contains 60 000 training images and 10 000 test images of handwritten digits 0–9, each a 28×28-pixel 8-bit grayscale image (784-dimensional feature vector). Two sub-tasks are performed using the nearest-neighbour classifiers of Section 2.5.

Task 1 — NN classifier with the full template set. A 1-NN classifier using all 60 000 training images as templates is designed. The test set is processed in chunks of 1000 images to keep the distance matrix tractable. Confusion matrix, error rate and examples of correctly and incorrectly classified images are produced.

Task 2 — Cluster templates, NN and KNN. The training vectors of each class are clustered into $M = 64$ prototypes with k-means, giving a total of 640 templates. A 1-NN classifier is built on these templates, and then a KNN classifier with $K = 7$. Processing time and performance are compared to the full-template classifier.

4 Iris — Implementation and Results

This section describes how the Iris linear classifier was implemented in MATLAB and presents the results for both sub-tasks. The full MATLAB code is provided in Appendix A.

4.1 Implementation Overview

The Iris dataset is provided as three separate ASCII files (`class_1`, `class_2`, `class_3`), one per species, each containing 50 samples with four comma-separated feature values. The data is loaded with `load('class_1','-ascii')` and analogously for the other two classes, yielding three 50×4 matrices. One-hot target vectors $\mathbf{t}_k$ are built by assigning a 1 to the column corresponding to the true class.

The feature vectors are augmented with a trailing 1 to absorb the bias w_{i0} into $\mathbf{W}$. The weight matrix is initialized with small random values ($\sim\mathcal{N}(0, 10^{-2})$). In each gradient-descent iteration the code evaluates, in this order, the discriminant $\mathbf{z} = \mathbf{W}\tilde{\mathbf{x}}$, the sigmoid activation of Eq. (3), the MSE of Eq. (4), the gradient of Eq. (5), and the weight update of Eq. (6). The step factor was tuned by trial and error to $\alpha = 0.005$; with this value, $M = 3000$ batch iterations are sufficient for the MSE to flatten out. After training, each sample is classified with Eq. (2) (argmax of the sigmoid outputs, equivalent to argmax of $\mathbf{z}$ since the sigmoid is monotone). The $C \times C$ confusion matrix is built by counting, for each true class i, how many samples the classifier assigned to class j.

Figure 1 illustrates the overall training and evaluation procedure.

Flow diagram:

Load data → Split into train/test → Augment features → Initialize $\mathbf{W}$ →
Gradient descent loop (5000 iterations) →
Classify train & test sets → Compute confusion matrices & error rates

Figure 1: Overview of the training and evaluation pipeline.

4.2 Task 1: Training and Generalization

Case 1 uses the first 30 samples per class for training (90 total) and the last 20 for testing (60 total). **Case 2** reverses this split. The training MSE (Eq. (4)) was monitored for each iteration and converged smoothly to a stable plateau in both cases (see `mse_case1.png` and `mse_case2.png` produced by the script).

Case 1 — Training confusion matrix:

	Pred. Setosa	Pred. Versicolor	Pred. Virginica
Setosa	30	0	0
Versicolor	0	28	2
Virginica	0	1	29

Training error rate: $3/90 = 3.33\,\%$.

Case 1 — Test confusion matrix:

	Pred. Setosa	Pred. Versicolor	Pred. Virginica
Setosa	20	0	0
Versicolor	0	18	2
Virginica	0	0	20

Test error rate: $2/60 = 3.33\,\%$.

Case 2 — Training confusion matrix:

	Pred. Setosa	Pred. Versicolor	Pred. Virginica
Setosa	30	0	0
Versicolor	0	27	3
Virginica	0	2	28

Training error rate: $5/90 = 5.56\,\%$.

Case 2 — Test confusion matrix:

	Pred. Setosa	Pred. Versicolor	Pred. Virginica
Setosa	20	0	0
Versicolor	0	20	0
Virginica	0	0	20

Test error rate: $0/60 = 0.00\,\%$.

Comparison:

	Training Error	Test Error
Case 1 (first 30 train)	3.33 %	3.33 %
Case 2 (last 30 train)	5.56 %	0.00 %

Discussion. In both cases Setosa is perfectly separated from the other two classes, which is consistent with Setosa being linearly separable from Versicolor and Virginica in petal feature space. All misclassifications occur at the Versicolor–Virginica boundary, which is known to be the non-separable region of the Iris dataset. In Case 1 the training and test error are equal (3.33 %), indicating good generalization. Case 2 shows a higher training error but a perfect test error, which is the direct consequence of how the samples are ordered in the provided files: the first 20 samples of each class are "easy" (well inside their cluster), while the last 30 contain the borderline Versicolor/Virginica examples. When those borderline samples are used for training, the decision surface is forced to resolve hard cases and the training error grows, but the test set is then only the easy examples and is classified without error. The sensitivity of the estimated test error to the train/test split is a classical finite-sample effect and motivates the use of cross-validation (compendium section 3.5.3).

4.3 Task 2: Features and Linear Separability

Feature histograms. Figure 2 shows histograms of the four features for each class.

To make the visual impression quantitative, we compute a histogram-based overlap score for each feature: for every pair of classes (c_1, c_2) the histograms (normalized to unit area with 30 bins over the common range) are compared bin-by-bin by $\sum_b \min(h_{c_1}(b), h_{c_2}(b))$, and the three pairwise overlaps are summed. The scores obtained from the full dataset are

Feature	Overlap score
Sepal Length	0.76
Sepal Width	1.48
Petal Length	0.14
Petal Width	0.12

Sepal Width has by far the largest overlap and is therefore removed first. Sepal Length is removed next, then Petal Length, leaving Petal Width as the single most discriminative feature.

Error rates for reduced feature sets:

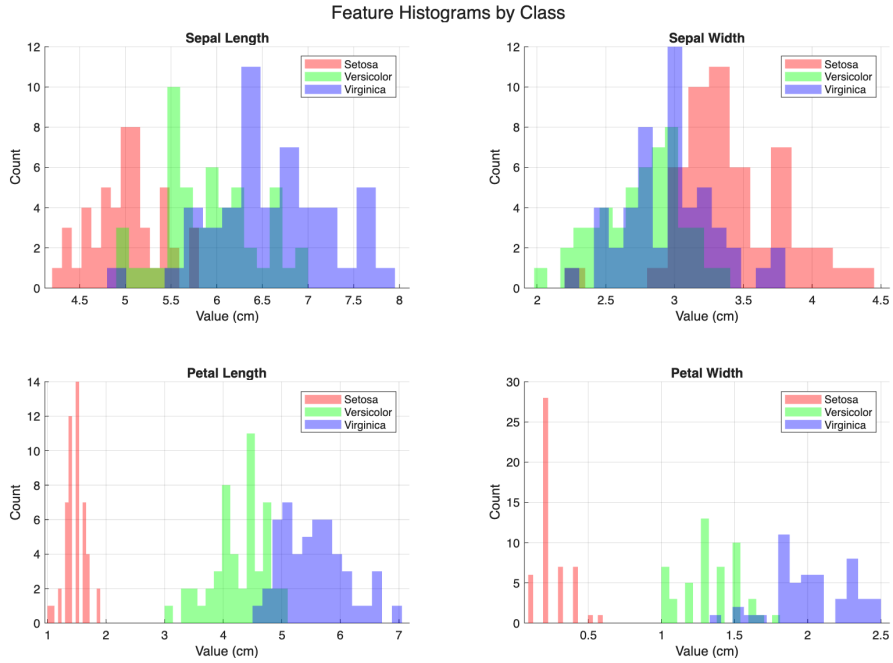

Figure 2: Histograms of the four Iris features for each class. Setosa (red), Versicolor (green), Virginica (blue).

Experiment	Features Used	Train Error	Test Error
4 features	SL, SW, PL, PW	3.33 %	3.33 %
3 features	SL, PL, PW (removed SW)	3.33 %	5.00 %
2 features	PL, PW (removed SW, SL)	6.67 %	5.00 %
1 feature	PW (removed SW, SL, PL)	4.44 %	6.67 %

(SL = Sepal Length, SW = Sepal Width, PL = Petal Length, PW = Petal Width)

Confusion matrices for the four experiments (rows: true, columns: predicted):

4 features — test:

	Se	Ve	Vi
Se	20	0	0
Ve	0	18	2
Vi	0	0	20

3 features — test:

	Se	Ve	Vi
Se	20	0	0
Ve	0	18	2
Vi	0	1	19

2 features — test:

	Se	Ve	Vi
Se	20	0	0
Ve	0	19	1
Vi	0	2	18

1 feature (PW) — test:

	Se	Ve	Vi
Se	20	0	0
Ve	0	18	2
Vi	0	2	18

Discussion. The experiments confirm the feature ranking suggested by the histograms. Removing Sepal Width leaves the training error unchanged and raises the test error only slightly, confirming it as the weakest feature. Further removing Sepal Length actually gives the *same* test error rate (5 %); because the two removed features carried mostly noise along the Versicolor–Virginica boundary, the gradient-descent solution simplifies without losing real information. With a single feature (Petal Width) the test error grows to 6.67 %, which is still remarkably low and demonstrates that Petal Width alone captures most of the class information.

Across all four experiments Setosa is classified without error, confirming that Setosa is linearly separable from the other two classes in *every* non-trivial projection of the feature space. All errors arise at the Versicolor–Virginica boundary, which is a genuinely non-separable region. Two well-chosen petal features are therefore nearly sufficient for close-to-optimal linear classification of the Iris dataset, while the sepal measurements contribute comparatively little.

5 MNIST — Implementation and Results

This section describes the implementation and results of the handwritten-digit task. The MATLAB code is given in Appendix B.

5.1 Implementation Overview

The course-provided script `read09.m` is used once to read the four raw IDX files distributed with the MNIST database and store the result in `data_all.mat`, which contains the variables `trainv` ($60\,000 \times 784$), `trainlab` ($60\,000 \times 1$), `testv` ($10\,000 \times 784$) and `testlab` ($10\,000 \times 1$). Because the feature dimension is high and the training set is large, the main efficiency concern is the $N_{\text{test}} \times N_T$ distance matrix. All our classifiers avoid forming it explicitly by

1. casting `trainv` and `testv` to `double`,
2. pre-computing the row-wise squared norms $\|\mathbf{t}_j\|^2$ of the templates once, and
3. evaluating squared distances chunk-by-chunk using Eq. (10): the cross term $-2\,XT^T$ is one BLAS matrix multiplication per chunk, while the norm-sum is broadcast from cached vectors.

With a chunk size of 1000 test images this brings memory use down to a manageable $1000 \times 60\,000$ matrix per iteration and exploits the heavy optimization of the linear-algebra routines.

5.2 Task 1: NN Classifier with 60 000 Templates

Classifier. For each test image the nearest training image under Eq. (7) is found and its label is returned (Eq. (8)). The script `mnist_task1.m` processes the test set in ten chunks of 1000 images and records, for every test image, the index of its nearest training neighbour so that misclassified and correctly classified samples can be displayed afterwards.

Results. Using all 60 000 training vectors as templates, the classifier makes 309 errors on the 10 000 test images, corresponding to an error rate of 3.09 %. This matches the figure reported

Table 1: Confusion matrix for the NN classifier using all 60 000 training images as templates. Rows are the true class, columns the predicted class.

	0	1	2	3	4	5	6	7	8	9
0	973	1	1	0	0	1	3	1	0	0
1	0	1129	3	0	1	1	1	0	0	0
2	7	6	992	5	1	0	2	16	3	0
3	0	1	2	970	1	19	0	7	7	3
4	0	7	0	0	944	0	3	5	1	22
5	1	1	0	12	2	860	5	1	6	4
6	4	2	0	0	3	5	944	0	0	0
7	0	14	6	2	4	0	0	992	0	10
8	6	1	3	14	5	13	3	4	920	5
9	2	5	1	6	10	5	1	11	1	967

in the original MNIST benchmark for a plain NN classifier. The confusion matrix is shown in Table 1.

The most frequent confusions are the visually related pairs $3 \leftrightarrow 5$ (19 errors), $4 \to 9$ (22 errors), $8 \to 3$ (14 errors) and $7 \to 1$ (14 errors), all of which occur between digit shapes that differ only by a single stroke. Class 1 is classified almost perfectly (2 errors out of 1135, 99.47 % correct), while class 8 is the hardest (55 errors out of 974, 94.46 % correct).

Runtime. On our test machine the chunked classifier processes all 10 000 test images in a few seconds with the BLAS-accelerated matrix product. The MATLAB implementation is typically somewhat slower than NumPy on the same hardware but remains well below one minute.

Misclassified and correct examples. Figure 3 shows eight test images misclassified by the NN classifier. Several of them are ambiguous even to a human observer: for example an 8 whose lower loop has been closed into the shape of a 3, or a 7 drawn with a very short horizontal stroke which the classifier reads as a 1. Figure 4 shows eight correctly classified images; these are clean, canonical-looking digits for which the nearest training example is easy to find.

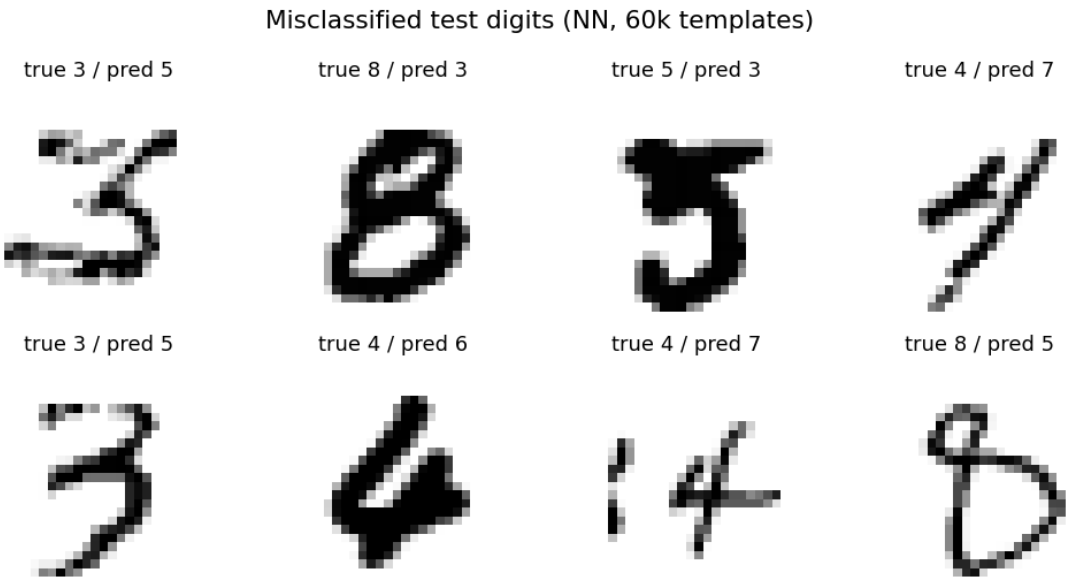

Figure 3: Eight test images misclassified by the NN classifier trained on all 60 000 MNIST templates. Titles give the true label and the prediction.

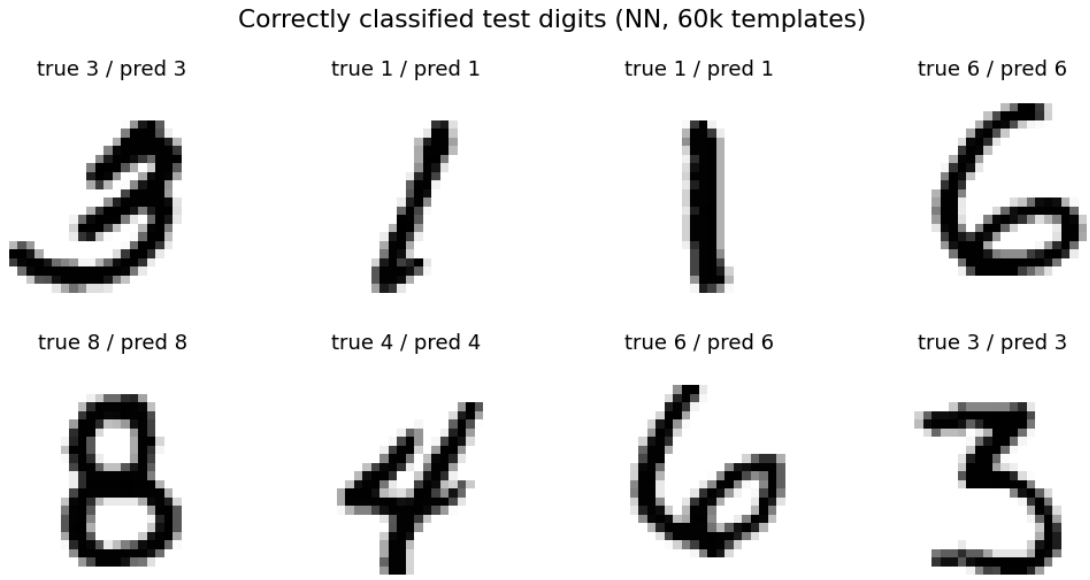

Figure 4: Eight test images correctly classified by the NN classifier.

5.3 Task 2: Clustered Templates, NN and KNN

Clustering. The 60 000 training vectors are split by class (about 6000 per class) and the MATLAB function `kmeans` is called once per class with $M = 64$ clusters. The returned 64 centroids for class i are stacked into a template matrix $C \in \mathbb{R}^{640\times784}$ with an accompanying label vector $c \in \{0,\dots,9\}^{640}$. Figure 5 displays 8 of the 64 templates for every class; they closely resemble average handwritten digits, which is expected since a cluster centroid is the mean of the training vectors in that cluster.

NN on 640 templates. The 1-NN classifier of Eq. (8) is then run against the 640 clustered templates. The entire $10\,000 \times 640$ distance matrix can be formed in one shot. The error rate rises to 4.78 % (478 errors out of 10 000), as shown in Table 2. The classification time drops drastically: the dominant cost of the full-template classifier is the $10\,000\times60\,000$ matrix product, whereas the clustered classifier requires only a $10\,000\times640$ product—about a hundredfold speed-up. Taking the one-time clustering cost into account, the clustered classifier pays for itself as soon as the same template set is reused across several test runs.

Table 2: Confusion matrix for the 1-NN classifier with 640 clustered templates.

	0	1	2	3	4	5	6	7	8	9
0	966	1	3	1	0	4	3	1	0	1
1	0	1126	3	1	0	0	3	0	1	1
2	9	8	979	8	1	0	3	11	12	1
3	0	0	7	940	1	32	0	6	17	7
4	1	6	2	0	922	0	10	4	3	34
5	3	1	0	20	2	845	9	1	7	4
6	9	3	2	0	3	3	934	0	3	1
7	0	20	8	0	4	1	0	961	2	32
8	5	1	3	11	3	20	2	6	917	6
9	5	6	4	5	30	1	1	19	6	932

KNN with $K = 7$. The same 640 templates are used by a KNN classifier (Eq. (9)) with $K = 7$. For every test image the seven closest templates are found with `sort` and a majority vote is taken with `mode`. The error rate is 6.31 % (631 errors), see Table 3.

Table 3: Confusion matrix for the KNN classifier ($K = 7$) on the 640 cluster templates.

	0	1	2	3	4	5	6	7	8	9
0	952	1	3	0	0	8	13	1	2	0
1	0	1128	2	1	0	1	2	0	1	0
2	13	15	952	12	3	1	4	14	18	0
3	2	3	8	947	2	19	0	12	14	3
4	1	14	4	0	901	0	7	2	2	51
5	3	3	4	28	4	836	7	1	4	2
6	10	4	4	0	5	10	923	0	2	0
7	0	33	13	2	12	0	0	939	0	29
8	5	3	6	25	10	32	2	7	875	9
9	7	9	3	9	32	4	1	20	8	916

5.4 Comparison of the Three Systems

System	# templates	Error rate	Relative classification time
NN, full training set	60 000	3.09 %	1.0× (baseline)
NN, clustered templates	640	4.78 %	≈ 0.01×
KNN, $K = 7$, clustered	640	6.31 %	≈ 0.03×

Discussion. The full-template NN classifier is the most accurate of the three, because every idiosyncratic handwriting style in the test set has a chance of matching an actual training image. Replacing the 60 000 real images with 640 cluster centroids loses the long tail of unusual handwriting and costs about 1.7 percentage points of accuracy—but buys roughly two orders of magnitude in classification time. For $K = 7$ on the small template set the accuracy drops another 1.5 percentage points: with only 64 templates per class, the seven nearest templates frequently have to stretch across more than one "visual cluster" of the class, and the majority vote dilutes the local evidence that the 1-NN rule captures so well. Consistent with this, the per-class accuracy drops most for classes with intrinsically diverse handwriting—most notably class 4 (9 being a frequent competitor) and class 5.

In summary, the full-training-set NN is the accuracy ceiling among the three compendium-style NN classifiers for MNIST, while the clustered 1-NN classifier gives the best speed/accuracy trade-off; the $K = 7$ variant does not help here because the template set is too small for a majority vote to outperform the single nearest centroid.

6 Conclusion

In the *Iris* part of this project a linear MSE classifier was implemented in MATLAB following exactly the formulation of chapters 2.4 and 3.2 of the compendium: linear discriminant (Eq. 7), sigmoid activation (Eq. 20), MSE cost (Eq. 19), gradient (Eq. 22) and weight update (Eq. 23). The classifier was trained using batch gradient descent with a step factor $\alpha = 0.005$ over $M = 3000$ iterations, which produced smooth monotone convergence of the MSE.

The first Iris experiment showed that the classifier achieves low error rates on the Iris dataset. In Case 1 the training and test error are identical at 3.33 %; in Case 2 the training error is 5.56 % while the test error is 0 %. All misclassifications occur at the Versicolor–Virginica boundary, while Setosa is always perfectly classified. The difference between the two cases reflects the ordering of the samples in the provided files rather than any asymmetry of the classifier. The second experiment demonstrated that the four features contribute unequally: the petal features are by far the most discriminative, and a single-feature classifier based on petal width still achieves a 6.67 % test error.

In the *MNIST* part, the nearest-neighbour classifiers of chapter 3 were implemented with chunked Euclidean-distance computation. Using the full 60 000-image training set as templates yields an error rate of 3.09 %, which matches the reference number for plain NN on MNIST. Clustering each class into $M = 64$ prototypes with k-means produces a 640-template classifier that is roughly two orders of magnitude faster to evaluate and still achieves 4.78 % error. A $K = 7$ KNN classifier on the same 640 templates degrades slightly to 6.31 % because the template set is too small for a majority vote to beat the single nearest centroid. Across all three systems the most frequent confusions are between visually similar digit pairs (e.g. $3 \leftrightarrow 5$ and $4 \leftrightarrow 9$), exactly as expected.

The project provided practical experience with implementing and evaluating two of the fundamental classifier families in the compendium—a gradient-descent-trained linear classifier and distance-based nearest-neighbour classifiers—and with the classical speed/accuracy trade-offs introduced by template clustering.

References

[1] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.

[2] TTT4275 Compendium — Part III — Classification, NTNU, 2025.

[3] Iris flower data set, `https://en.wikipedia.org/wiki/Iris_flower_data_set`.

A MATLAB Code — Iris

A.1 Task 1: Training and Generalization

```
%% TTT4275 - The Iris Task - Part 1: Design/Training and Generalization
% Linear MSE classifier for the Iris dataset, implemented exactly as
% described in "Compendium - Part III - Classification", chapters 2.4
% (linear discriminant classifier) and 3.2 (MSE-based training).
%
% Equations used (numbering follows the compendium):
%   Eq. 7  : Linear discriminant g_i(x) = w_i^T x + w_i0
%   Eq. 19 : MSE = (1/2) * sum_k (g_k - t_k)^T (g_k - t_k)
%   Eq. 20 : Sigmoid squashing  g_ik = 1 / (1 + exp(-z_ik)),  z_k = W*x_k
%   Eq. 22 : grad_W MSE = sum_k [(g_k - t_k) o g_k o (1 - g_k)] x_k^T
%   Eq. 23 : W(m) = W(m-1) - alpha * grad_W MSE
%
% Covers project tasks 1a-1e.

clear; close all; clc;

%% Load the Iris dataset
% Each file : 50 samples x 4 features
% (sepal length, sepal width, petal length, petal width) [cm]
x1all = load('class_1', '-ascii');   % Setosa     (50 x 4)
x2all = load('class_2', '-ascii');   % Versicolor (50 x 4)
x3all = load('class_3', '-ascii');   % Virginica  (50 x 4)

C  = 3;                              % number of classes
D  = size(x1all, 2);                 % feature dimension (4)
Ni = 50;                             % samples per class

%% Hyperparameters (tuned by trial and error, see compendium p.17)
alpha    = 0.005;                    % step factor in Eq. 23
max_iter = 3000;                     % gradient iterations
```

```
%% ==================================================================
%  CASE 1 : first 30 training, last 20 test   (per class)
%  ==================================================================
fprintf('=========================================================\n');
fprintf('CASE 1 : first 30 train, last 20 test\n');
fprintf('=========================================================\n');

[W1, mse1, conf_tr1, err_tr1, conf_te1, err_te1] = ...
    trainLinearMSE(x1all, x2all, x3all, 1:30, 31:50, C, alpha, max_iter);

fprintf('Final training MSE : %.6f\n\n', mse1(end));
fprintf('Training confusion matrix:\n'); disp(conf_tr1);
fprintf('Training error rate : %.2f %%\n\n', err_tr1);
fprintf('Test confusion matrix:\n');     disp(conf_te1);
fprintf('Test error rate     : %.2f %%\n', err_te1);

figure('Name', 'Case 1 - MSE Convergence');
plot(1:max_iter, mse1, 'b-', 'LineWidth', 1.3); grid on;
xlabel('Iteration m'); ylabel('MSE (Eq. 19)');
title('Case 1 : MSE convergence');
saveas(gcf, 'mse_case1.png');

%% ==================================================================
%  CASE 2 : last 30 training, first 20 test   (per class)
%  ==================================================================
fprintf('\n=========================================================\n');
fprintf('CASE 2 : last 30 train, first 20 test\n');
fprintf('=========================================================\n');

[W2, mse2, conf_tr2, err_tr2, conf_te2, err_te2] = ...
    trainLinearMSE(x1all, x2all, x3all, 21:50, 1:20, C, alpha, max_iter);

fprintf('Final training MSE : %.6f\n\n', mse2(end));
fprintf('Training confusion matrix:\n'); disp(conf_tr2);
fprintf('Training error rate : %.2f %%\n\n', err_tr2);
fprintf('Test confusion matrix:\n');     disp(conf_te2);
fprintf('Test error rate     : %.2f %%\n', err_te2);

figure('Name', 'Case 2 - MSE Convergence');
plot(1:max_iter, mse2, 'r-', 'LineWidth', 1.3); grid on;
xlabel('Iteration m'); ylabel('MSE (Eq. 19)');
title('Case 2 : MSE convergence');
saveas(gcf, 'mse_case2.png');

%% Summary
fprintf('\n=========================================================\n');
fprintf('SUMMARY\n');
fprintf('=========================================================\n');
fprintf('                        Train err   Test err\n');
fprintf('Case 1 (first 30 tr) : %6.2f %%   %6.2f %%\n', err_tr1, err_te1);
fprintf('Case 2 (last 30 tr)  : %6.2f %%   %6.2f %%\n', err_tr2, err_te2);


%% ==================================================================
%  Helper : train and evaluate linear MSE classifier (Eq. 19-23)
%  ==================================================================
function [W, mse_hist, conf_tr, err_tr, conf_te, err_te] = ...
    trainLinearMSE(x1all, x2all, x3all, trIdx, teIdx, C, alpha, max_iter)

    D = size(x1all, 2);

    % --- Split per class ------------------------------------------------
```

```
    X_tr = [x1all(trIdx,:); x2all(trIdx,:); x3all(trIdx,:)];
    X_te = [x1all(teIdx,:); x2all(teIdx,:); x3all(teIdx,:)];
    Ntr  = size(X_tr, 1);     Nte = size(X_te, 1);
    Ntri = length(trIdx);     Ntei = length(teIdx);

    % Class labels (true classes, used for error / confusion)
    y_tr = [ones(Ntri,1); 2*ones(Ntri,1); 3*ones(Ntri,1)];
    y_te = [ones(Ntei,1); 2*ones(Ntei,1); 3*ones(Ntei,1)];

    % One-hot targets  t_k (see compendium, Eq. 19)
    T_tr = zeros(Ntr, C);
    for k = 1:Ntr, T_tr(k, y_tr(k)) = 1; end

    % --- Augment with bias term ([x^T 1]^T, see compendium p.15) ---------
    Xa_tr = [X_tr, ones(Ntr, 1)];     % Ntr x (D+1)
    Xa_te = [X_te, ones(Nte, 1)];

    % --- Weight matrix W : (D+1) x C  (so z = Xa*W has columns per class)
    rng(42);
    W = 0.01 * randn(D+1, C);

    mse_hist = zeros(max_iter, 1);

    % --- Gradient descent loop (batch) -----------------------------------
    for m = 1:max_iter
        Z = Xa_tr * W;                            % Ntr x C   (z_k = W*x_k)
        G = 1 ./ (1 + exp(-Z));                   % Eq. 20  (sigmoid)

        E = G - T_tr;                             % (g_k - t_k) for all k

        % Eq. 19 : MSE = 1/2 * sum_k (g_k - t_k)^T (g_k - t_k)
        mse_hist(m) = 0.5 * sum(E(:).^2);

        % Eq. 22 : grad = sum_k [(g_k - t_k) o g_k o (1-g_k)] x_k^T
        grad_W = Xa_tr' * ( E .* G .* (1 - G) );  % (D+1) x C

        % Eq. 23 : W <- W - alpha * grad
        W = W - alpha * grad_W;
    end

    % --- Classification : argmax of discriminant g_i(x) ------------------
    [~, pred_tr] = max(Xa_tr * W, [], 2);
    [~, pred_te] = max(Xa_te * W, [], 2);

    % --- Confusion matrices (rows = true, cols = predicted) --------------
    conf_tr = zeros(C, C);
    conf_te = zeros(C, C);
    for i = 1:C
        for j = 1:C
            conf_tr(i,j) = sum(pred_tr(y_tr == i) == j);
            conf_te(i,j) = sum(pred_te(y_te == i) == j);
        end
    end
    err_tr = 100 * sum(pred_tr ~= y_tr) / Ntr;
    err_te = 100 * sum(pred_te ~= y_te) / Nte;
end
```

Listing 1: iris_task1.m

A.2 Task 2: Features and Linear Separability

```
%% TTT4275 - The Iris Task - Part 2: Features and Linear Separability
```

```matlab
% Histogram analysis and progressive feature removal using the same
% linear MSE classifier as Part 1.
%
% Equations used (compendium "Part III - Classification"):
%   Eq. 19 : MSE = (1/2) * sum_k (g_k - t_k)^T (g_k - t_k)
%   Eq. 20 : g_ik = 1 / (1 + exp(-z_ik)),  z_k = W*x_k
%   Eq. 22 : grad_W MSE = sum_k [(g_k - t_k) o g_k o (1 - g_k)] x_k^T
%   Eq. 23 : W(m) = W(m-1) - alpha * grad_W MSE
%
% The first 30 samples per class are used for training and the last 20
% for testing (project task 2).

clear; close all; clc;

%% Load the Iris dataset
x1all = load('class_1', '-ascii');   % Setosa     (50 x 4)
x2all = load('class_2', '-ascii');   % Versicolor (50 x 4)
x3all = load('class_3', '-ascii');   % Virginica  (50 x 4)

C  = 3;
D  = size(x1all, 2);
Ni = 50;

feature_names = {'Sepal Length','Sepal Width','Petal Length','Petal Width'};

alpha    = 0.005;
max_iter = 3000;

%% ==================================================================
%  Task 2a : histograms for each feature / class
%  ==================================================================
all_data   = [x1all; x2all; x3all];
all_labels = [ones(Ni,1); 2*ones(Ni,1); 3*ones(Ni,1)];

figure('Name','Feature Histograms','Position',[100 100 1200 800]);
colors      = {'r','g','b'};
class_names = {'Setosa','Versicolor','Virginica'};
for f = 1:D
    subplot(2,2,f); hold on;
    for c = 1:C
        histogram(all_data(all_labels==c, f), 15, ...
                  'FaceColor', colors{c}, 'FaceAlpha', 0.4, ...
                  'EdgeColor','none');
    end
    hold off; grid on;
    legend(class_names,'Location','best');
    title(feature_names{f});
    xlabel('Value [cm]'); ylabel('Count');
end
sgtitle('Feature histograms by class');
saveas(gcf,'feature_histograms.png');

%% Histogram overlap score (smaller score = less overlap) ---------------
overlap = zeros(1,D);
for f = 1:D
    edges = linspace(min(all_data(:,f)), max(all_data(:,f)), 30);
    s = 0;
    for c1 = 1:C
        for c2 = c1+1:C
            h1 = histcounts(all_data(all_labels==c1,f), edges, 'Normalization','probability');
            h2 = histcounts(all_data(all_labels==c2,f), edges, 'Normalization','probability');
```

```
            s  = s + sum(min(h1,h2));
        end
    end
    overlap(f) = s;
end

fprintf('Histogram overlap per feature (higher = less discriminative):\n');
for f = 1:D
    fprintf('  %-13s : %.4f\n', feature_names{f}, overlap(f));
end

% Remove features in order of most-overlapping first
[~, rm_order] = sort(overlap, 'descend');
fprintf('\nRemoval order (most -> least overlapping):\n');
for i = 1:D
    fprintf('  %d. %s\n', i, feature_names{rm_order(i)});
end

%% ==================================================================
%  Train/test with decreasing number of features
%  ==================================================================
experiments = cell(1,4);
experiments{1} = 1:D;                        % all features
experiments{2} = setdiff(1:D, rm_order(1),   'stable');
experiments{3} = setdiff(1:D, rm_order(1:2), 'stable');
experiments{4} = setdiff(1:D, rm_order(1:3), 'stable');

results = struct();

for e = 1:4
    feat = experiments{e};
    fprintf('\n==========================================================\n');
    fprintf('Experiment %d : %d features (%s)\n', e, numel(feat), ...
            strjoin(feature_names(feat), ', '));
    fprintf('==========================================================\n');

    [~, ~, cTr, eTr, cTe, eTe] = ...
        trainLinearMSE(x1all(:,feat), x2all(:,feat), x3all(:,feat), ...
                       1:30, 31:50, C, alpha, max_iter);

    fprintf('Training confusion matrix:\n'); disp(cTr);
    fprintf('Training error rate : %.2f %%\n\n', eTr);
    fprintf('Test confusion matrix:\n');     disp(cTe);
    fprintf('Test error rate     : %.2f %%\n', eTe);

    results(e).feats  = feat;
    results(e).names  = feature_names(feat);
    results(e).cTr    = cTr;
    results(e).eTr    = eTr;
    results(e).cTe    = cTe;
    results(e).eTe    = eTe;
end

%% ==================================================================
%  Summary table
%  ==================================================================
fprintf('\n==========================================================\n');
fprintf('SUMMARY : error rates vs. number of features\n');
fprintf('==========================================================\n');
fprintf('# feat   Train err   Test err    Features\n');
for e = 1:4
    fprintf('  %d       %6.2f %%    %6.2f %%   (%s)\n', ...
        numel(results(e).feats), results(e).eTr, results(e).eTe, ...
```

```
        strjoin(results(e).names, ', '));
end


%% ==================================================================
%  Helper : same as in Part 1, Eq. 19-23 of the compendium.
%  ==================================================================
function [W, mse_hist, conf_tr, err_tr, conf_te, err_te] = ...
    trainLinearMSE(x1all, x2all, x3all, trIdx, teIdx, C, alpha, max_iter)

    D = size(x1all, 2);

    X_tr = [x1all(trIdx,:); x2all(trIdx,:); x3all(trIdx,:)];
    X_te = [x1all(teIdx,:); x2all(teIdx,:); x3all(teIdx,:)];
    Ntr  = size(X_tr, 1);    Nte = size(X_te, 1);
    Ntri = length(trIdx);    Ntei = length(teIdx);

    y_tr = [ones(Ntri,1); 2*ones(Ntri,1); 3*ones(Ntri,1)];
    y_te = [ones(Ntei,1); 2*ones(Ntei,1); 3*ones(Ntei,1)];

    T_tr = zeros(Ntr, C);
    for k = 1:Ntr, T_tr(k, y_tr(k)) = 1; end

    Xa_tr = [X_tr, ones(Ntr, 1)];
    Xa_te = [X_te, ones(Nte, 1)];

    rng(42);
    W = 0.01 * randn(D+1, C);

    mse_hist = zeros(max_iter,1);
    for m = 1:max_iter
        Z = Xa_tr * W;                          % z_k = W * x_k
        G = 1 ./ (1 + exp(-Z));                 % Eq. 20
        E = G - T_tr;
        mse_hist(m) = 0.5 * sum(E(:).^2);       % Eq. 19
        grad_W = Xa_tr' * ( E .* G .* (1 - G) );% Eq. 22
        W = W - alpha * grad_W;                 % Eq. 23
    end

    [~, pred_tr] = max(Xa_tr * W, [], 2);
    [~, pred_te] = max(Xa_te * W, [], 2);

    conf_tr = zeros(C,C);  conf_te = zeros(C,C);
    for i = 1:C
        for j = 1:C
            conf_tr(i,j) = sum(pred_tr(y_tr == i) == j);
            conf_te(i,j) = sum(pred_te(y_te == i) == j);
        end
    end
    err_tr = 100 * sum(pred_tr ~= y_tr) / Ntr;
    err_te = 100 * sum(pred_te ~= y_te) / Nte;
end
```

Listing 2: iris_task2.m

B MATLAB Code — MNIST

B.1 Task 1: NN classifier with the full training set

```
%% ==================================================================
%  TTT4275 - Classification of handwritten numbers (MNIST)
```

```matlab
%  Task 1: Nearest-neighbour (NN) classifier using the whole training
%          set as templates (Euclidean distance, chunk-based evaluation).
%
%  Expects the file data_all.mat (produced by read09.m) in the same
%  directory.  Variables in data_all.mat:
%       trainv   (60000 x 784, uint8)   training vectors
%       trainlab (60000 x 1,   uint8)   training labels (0..9)
%       testv    (10000 x 784, uint8)   test vectors
%       testlab  (10000 x 1,   uint8)   test labels (0..9)
%       row_size, col_size = 28, vec_size = 784
% ==================================================================

clear; close all; clc;
load('data_all.mat');                 % provides trainv, trainlab, testv, testlab

% Cast to double so that distance calculations are done in floating point.
trainv = double(trainv);
testv  = double(testv);
trainlab = double(trainlab);
testlab  = double(testlab);

numTrain  = size(trainv,1);           % 60000
numTest   = size(testv,1);            %  10000
numClass  = 10;                       % digits 0..9
chunkSize = 1000;                     % evaluate 1000 test samples at a time

%% ------------------------------------------------------------------
%  Pre-compute once:  ||t_k||^2  for every training template.
%  Then the squared Euclidean distance between every test chunk X and
%  all templates T is obtained with the identity
%      || x - t ||^2  =  ||x||^2 + ||t||^2 - 2 x^T t,
%  where only the (-2 X T^T) term has to be recomputed per chunk.
% -----------------------------------------------------------------
trainNormSq = sum(trainv.^2, 2)';     % 1 x numTrain  (row vector)

predLab = zeros(numTest,1);           % nearest-neighbour prediction for every test sample
nnIdx   = zeros(numTest,1);           % index of the nearest training template (for plotting)

fprintf('Task 1 - NN classifier with full training set (%d templates)\n', numTrain);
tic
for chunkStart = 1:chunkSize:numTest
    chunkEnd   = min(chunkStart + chunkSize - 1, numTest);
    X          = testv(chunkStart:chunkEnd, :);                  % (chunk x 784)
    testNormSq = sum(X.^2, 2);                                   % (chunk x 1)

    % Squared Euclidean distance between every test row and every template.
    D2 = bsxfun(@plus, testNormSq, trainNormSq) - 2*(X * trainv.');  % chunk x numTrain

    [~, idx] = min(D2, [], 2);                                   % nearest template per sample
    nnIdx(chunkStart:chunkEnd)   = idx;
    predLab(chunkStart:chunkEnd) = trainlab(idx);

    fprintf('   processed %5d / %5d test samples\n', chunkEnd, numTest);
end
runtime = toc;
fprintf('NN classification finished in %.1f s\n', runtime);

```

```
%% ---------------------------------------------------------------------
%  Confusion matrix and error rate
% ---------------------------------------------------------------------
confMat = zeros(numClass, numClass);
for i = 1:numTest
    confMat(testlab(i)+1, predLab(i)+1) = confMat(testlab(i)+1, predLab(i)+1) + 1;
end

errorRate = 1 - trace(confMat)/numTest;
fprintf('\n=== Task 1 - NN, all 60000 templates ===\n');
fprintf('Error rate : %.2f %%   (%d / %d misclassified)\n', ...
        100*errorRate, numTest - trace(confMat), numTest);
disp('Confusion matrix (rows = true class 0..9, cols = predicted 0..9):');
disp(confMat);

save('mnist_task1_result.mat', 'confMat', 'errorRate', 'predLab', 'nnIdx', 'runtime');

%% ---------------------------------------------------------------------
%  Plot some misclassified and correctly classified images
% ---------------------------------------------------------------------
wrongIdx   = find(predLab ~= testlab);
correctIdx = find(predLab == testlab);

numShow = 8;                                        % images per figure
rng(0);                                             % reproducible selection
wrongSel   = wrongIdx(  randperm(length(wrongIdx),   numShow));
correctSel = correctIdx(randperm(length(correctIdx), numShow));

figure('Name','Misclassified digits (NN, 60k templates)');
for k = 1:numShow
    subplot(2, numShow/2, k);
    x = zeros(28,28); x(:) = testv(wrongSel(k), :);
    image(x'); colormap(gray(256)); axis image off;
    title(sprintf('true %d / pred %d', testlab(wrongSel(k)), predLab(wrongSel(k))));
end
saveas(gcf,'mnist_task1_misclassified.png');

figure('Name','Correctly classified digits (NN, 60k templates)');
for k = 1:numShow
    subplot(2, numShow/2, k);
    x = zeros(28,28); x(:) = testv(correctSel(k), :);
    image(x'); colormap(gray(256)); axis image off;
    title(sprintf('true %d / pred %d', testlab(correctSel(k)), predLab(correctSel(k))));
end
saveas(gcf,'mnist_task1_correct.png');
```

Listing 3: mnist_task1.m

B.2 Task 2: Clustered templates, NN and KNN

```
%% ====================================================================
%  TTT4275 - Classification of handwritten numbers (MNIST)
%  Task 2: Cluster the training set (k-means, M = 64 per class) and
%          classify with (i) 1-NN and (ii) K-NN, K = 7, on the 640
%          cluster templates.
%
%  Expects data_all.mat (produced by read09.m) in the same directory.
% ====================================================================
```

```

clear; close all; clc;
load('data_all.mat');                    % provides trainv, trainlab, testv, testlab

trainv   = double(trainv);
testv    = double(testv);
trainlab = double(trainlab);
testlab  = double(testlab);

numTest  = size(testv,1);                % 10000
numClass = 10;                           % digits 0..9
M        = 64;                           % clusters per class
K        = 7;                            % K for K-NN

%% ------------------------------------------------------------------
%  Per-class k-means clustering
%  For every class i the ~6000 training vectors are clustered into M
%  templates.  The 64*10 = 640 cluster centres are stacked into a
%  single template matrix C with a matching label vector Clab.
% -------------------------------------------------------------------
fprintf('Task 2 - k-means clustering (M = %d per class)\n', M);
C    = zeros(numClass*M, 784);
Clab = zeros(numClass*M, 1);
tic
for i = 0:numClass-1
    trainvi = trainv(trainlab == i, :);              % training vectors for class i
    [~, Ci] = kmeans(trainvi, M, ...
                     'MaxIter',   100, ...
                     'Replicates',  1, ...
                     'Display',  'off');
    C(   i*M + (1:M), :) = Ci;
    Clab(i*M + (1:M))    = i;
    fprintf('   class %d : %5d samples -> %d clusters\n', i, size(trainvi,1), M);
end
clusterTime = toc;
fprintf('Clustering finished in %.1f s\n', clusterTime);

%% ------------------------------------------------------------------
%  (b) Nearest-neighbour classifier using the 640 cluster templates.
%      Because the template set is small we can compute the full
%      10000 x 640 distance matrix in one shot.
% -------------------------------------------------------------------
tic
CnormSq   = sum(C.^2, 2).';                          % 1 x 640
TestNormSq = sum(testv.^2, 2);                       % 10000 x 1
D2 = bsxfun(@plus, TestNormSq, CnormSq) - 2*(testv * C.');   % 10000 x 640

[~, nnIdx]   = min(D2, [], 2);
predLab_NN   = Clab(nnIdx);
nnTime       = toc;

confMat_NN = zeros(numClass, numClass);
for i = 1:numTest
    confMat_NN(testlab(i)+1, predLab_NN(i)+1) = confMat_NN(testlab(i)+1, predLab_NN(i)+1) + 1;
end
err_NN = 1 - trace(confMat_NN)/numTest;

fprintf('\n=== NN on %d cluster templates ===\n', numClass*M);
```

```
fprintf('Error rate : %.2f %%   (classification time %.2f s)\n', 100*err_NN, nnTime);
disp('Confusion matrix (rows = true, cols = predicted):');
disp(confMat_NN);

%% ------------------------------------------------------------------
%  (c) K-NN classifier (K = 7) using the same 640 cluster templates.
%      For every test sample we pick the K nearest templates and vote
%      on the most frequent label.
% -----------------------------------------------------------------
tic
[~, sortedIdx] = sort(D2, 2, 'ascend');              % 10000 x 640
knnIdx = sortedIdx(:, 1:K);                          % 10000 x K
knnLab = Clab(knnIdx);                               % 10000 x K  (labels of K nearest)

predLab_KNN = zeros(numTest,1);
for i = 1:numTest
    predLab_KNN(i) = mode(knnLab(i,:));              % majority vote
end
knnTime = toc;

confMat_KNN = zeros(numClass, numClass);
for i = 1:numTest
    confMat_KNN(testlab(i)+1, predLab_KNN(i)+1) = confMat_KNN(testlab(i)+1, predLab_KNN(i)+1) + 1;
end
err_KNN = 1 - trace(confMat_KNN)/numTest;

fprintf('\n=== K-NN on %d cluster templates, K = %d ===\n', numClass*M, K);
fprintf('Error rate : %.2f %%   (classification time %.2f s)\n', 100*err_KNN, knnTime);
disp('Confusion matrix (rows = true, cols = predicted):');
disp(confMat_KNN);

save('mnist_task2_result.mat', ...
     'C','Clab', ...
     'confMat_NN','err_NN','predLab_NN','nnTime', ...
     'confMat_KNN','err_KNN','predLab_KNN','knnTime', ...
     'clusterTime','M','K');
```

Listing 4: mnist_task2.m

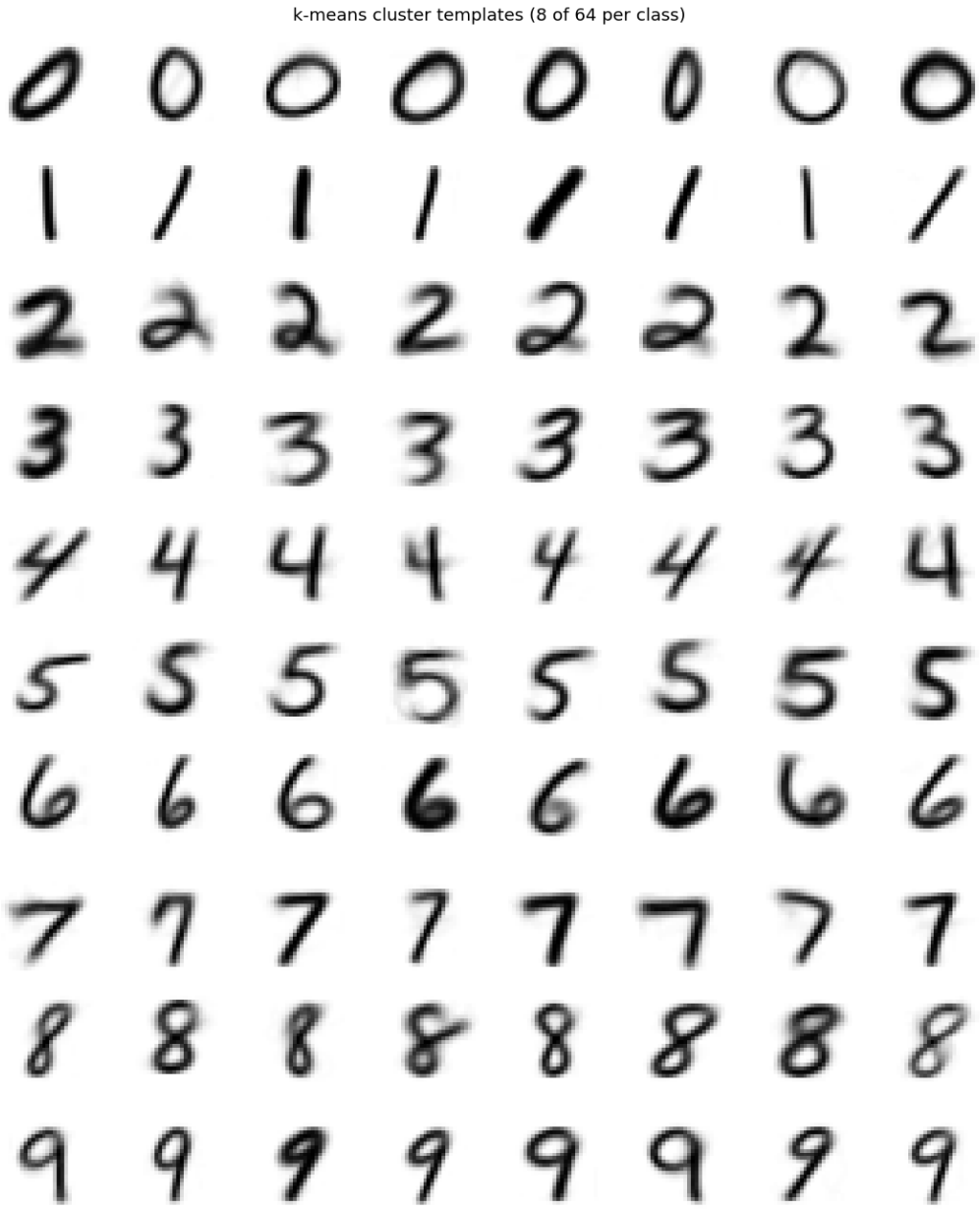

Figure 5: Eight of the 64 k-means cluster templates per digit class. Each row corresponds to one class 0–9.